

Guide builder — mise en vulnérabilité, chaînes et flags

DOCUMENT

Guide builder

PROMOTION

Master 1



Sommaire

Guide builder — mise en vulnérabilité, chaînes et flags	3
1. Quotas à respecter	3
2. Méthode en 6 étapes	4
Étape 1 — Cartographier votre propre SI	4
Étape 2 — Choisir les trois points d'entrée (N1)	4
Étape 3 — Construire les chaînes	4
Étape 4 — Verrouiller la profondeur	5
Étape 5 — Habiller	5
Étape 6 — Placer et valider les flags	5
3. Défauts de conception volontaires (recommandé)	5
4. Ce que la cellule défense doit pouvoir détecter	6
5. Recette avant livraison	6

Guide builder — mise en vulnérabilité, chaînes et flags

Document **commun à tous les groupes**, quel que soit le cahier des charges choisi. Il explique *comment* transformer un SI d'entreprise conforme au CDC en terrain d'exercice.

Ce guide est commun à toute la promotion, et le catalogue des profils aussi. Restent secrets : **le cahier des charges qui vous a été attribué**, et surtout ce que vous en faites — quels services vous rendez vulnérables, avec quoi, et comment vous les enchaînez.

1. Quotas à respecter

Exigence	Règle
Services de la tranche ferme du CDC déployés	tous (16 ou 17 selon le CDC)
Services vulnérables	12 (2 × 6 membres)
Points d'entrée N1	3, et seulement 3 — les seules vulnérabilités atteignables depuis l'extérieur
Vulnérabilités N2 + N3	les 9 autres , répartition libre entre les deux niveaux
Chaînes complètes N1 → N2 → N3	2
Flags placés	1 par service vulnérable + 1 par pivot notable (12 minimum)

La règle structurante est celle des N1 : **les trois points d'entrée sont la seule façon d'entrer**. Tout le reste du SI — les 9 autres vulnérabilités — doit exiger un pivot. Si un quatrième service est exploitable directement depuis l'extérieur, vous avez quatre N1 et vous êtes hors quota, quelle que soit votre intention.

En revanche, une fois à l'intérieur, **vous êtes libres** : 5 N2 et 4 N3, ou 3 N2 et 6 N3, ou toute autre répartition, du moment que deux chaînes vont bien de N1 à N3.

Le compte est serré et c'est voulu : votre CDC demande 16 ou 17 services, dont l'OPNsense qui est déjà fourni. Il vous reste donc 15 ou 16 services à déployer, dont **12 doivent porter une vulnérabilité**. Autrement dit, 3 ou 4 services seulement resteront sains — choisissez-les utiles : un service sain qui contient les credentials du service suivant vaut mieux qu'un service sain décoratif.

Répartition indicative du travail à 6 : deux personnes sur l'infrastructure (réseaux Docker, publication et filtrage OPNsense, règles Suricata), deux sur les services publiés et leurs chaînes, deux sur les services internes, les données métier et la documentation. Chacun est responsable de bout en bout de ses services : déploiement, habillage, exploit validé, flag, chapitre de documentation.

Un service vulnérable de plus vaut mieux qu'un service vulnérable de trop : un exploit qui ne fonctionne pas le jour J **compte comme un service manquant**.

2. Méthode en 6 étapes

Étape 1 — Cartographier votre propre SI

Listez les services du CDC et classez-les en trois familles :

- **exposés** (publiés par l'OPNsense vers l'hôte Docker) → candidats N1 ;
- **internes accessibles depuis un exposé** (même réseau Docker, ou joignables par un conteneur passerelle) → candidats N2 ;
- **cœur du SI** (données, zone spécifique, administration) → candidats N3.

Rappel de mise en œuvre : les zones du CDC sont des **réseaux Docker** (`net-dmz`, `net-srv`, `net-data`, `net-users`, `net-admin`, `net-spec`). Ce qui sépare un N1 d'un N2, c'est l'appartenance à des réseaux différents — pas une simple différence de port.

Étape 2 — Choisir les trois points d'entrée (N1)

Trois, exactement. Un point d'entrée doit être exploitable **avec les seuls éléments du document attaquant** : URL publique + éventuellement un compte de bas privilège.

- Prenez-les parmi vos services publiés (votre CDC en compte 4 ou 5) : cela vous laisse au moins un service publié qui reste sain.
- Évitez de mettre vos N1 sur la même technologie : si l'attaquant trouve l'un, il ne doit pas déduire les autres.
- Chaque N1 doit ouvrir sur quelque chose : deux d'entre eux amorcent vos deux chaînes, le troisième peut mener à un pivot latéral ou à un flag isolé.
- **Vérifiez qu'il n'y en a pas un quatrième par accident** : un service publié laissé vulnérable « pour faire joli », une page d'administration oubliée, un port publié en trop. C'est l'erreur la plus fréquente et elle casse le quota.

Étape 3 — Construire les chaînes

Une chaîne se construit **à l'envers** : partez de l'objectif (une donnée métier critique), remontez jusqu'au point d'entrée.

Le maillon d'une chaîne, ce n'est pas la vulnérabilité — c'est **l'information laissée derrière**. Après chaque compromission, l'attaquant doit trouver quelque chose qui lui ouvre l'étape suivante :

Ce que l'attaquant compromet	Ce qu'il doit y trouver
Un serveur web	fichier de configuration avec la chaîne de connexion à la base
Un partage de fichiers	script de sauvegarde contenant un mot de passe de service
Une messagerie / un ticketing	échange avec un prestataire contenant une procédure et des identifiants
Un wiki / une GED	page de procédure, PV de recette, marché public avec des identifiants
Un dépôt Git	secret dans l'historique, variable de pipeline, clé SSH
Une base de données	table d'utilisateurs, mots de passe réutilisés ailleurs
Un outil de supervision	inventaire complet du SI, credentials d'agents
Un cache / une file	jetons de session, messages applicatifs

Règle d'or : **chaque credential trouvé doit ouvrir exactement une porte**. Un mot de passe qui ouvre tout casse la progressivité ; un credential qui n'ouvre rien frustre inutilement.

Étape 4 — Verrouiller la profondeur

C'est ici que se joue le respect du quota. Un N2 ne doit **pas** être joignable depuis l'extérieur, un N3 ne doit **pas** être joignable depuis la DMZ. Concrètement : pas de `ports` publiés sur les services non exposés, et pas de conteneur attaché à deux réseaux sans raison documentée.

Vérifiez-le par le test suivant, **obligatoire** avant la livraison :

```
depuis l'extérieur (poste attaquant) : nmap -Pn -p- <ip_public> -> uniquement les ports publiés
depuis un shell dans le conteneur N1 : nmap -Pn -p- <ip_N2> -> le service repond
                                      nmap -Pn -p- <ip_N3> -> ferme
depuis un shell dans le conteneur N2 : nmap -Pn -p- <ip_N3> -> le service repond
```

Astuce : `docker network inspect net-srv` donne les adresses des conteneurs d'un réseau ; c'est aussi ce que fera l'attaquant une fois dans un conteneur.

Cette matrice de tests est reproduite dans le document pédagogique.

Étape 5 — Habiller

Un service Vulhub brut se reconnaît en trois secondes et ne rapporte aucun point de réalisme. Travail attendu sur **chaque** service :

- nom de domaine et certificat auto-signé aux couleurs de l'entreprise ;
- page d'accueil, thème, logo, pied de page avec mentions légales ;
- comptes cohérents avec l'organigramme du CDC ;
- données métier (voir les volumes exigés par le CDC) ;
- traces d'usage : historique, documents, tickets, e-mails, commits.

Vous pouvez modifier `docker-compose.yml` et `Dockerfile`, ajouter un reverse proxy devant, changer les ports, injecter du contenu au démarrage (script d'init, `COPY` dans le `Dockerfile`, dump SQL monté en volume). **Interdit** : patcher la vulnérabilité, filtrer l'URL vulnérable, ajouter un WAF bloquant, exiger une authentification non fournie sur un service N1.

Étape 6 — Placer et valider les flags

Le flag matérialise la preuve d'exploitation. Il doit être :

- **atteignable uniquement par le chemin prévu** (pas dans un répertoire listable, pas dans une image publique, pas dans un dépôt Git accessible) ;
- **au bon niveau de privilège** : si l'exploit donne un shell `www-data`, un flag dans `/root/flag.txt` exige en plus une élévation — c'est un choix, pas un accident ;
- **persistant** : un flag écrit à la main dans un conteneur disparaît au prochain `docker compose down`. Placez-le via le `Dockerfile`, un script d'init ou un volume ;
- **issu du CSV fourni** — n'inventez jamais un flag.

Emplacements conventionnels : voir le *Socle technique*, §7.

3. Défauts de conception volontaires (recommandé)

Un SI d'entreprise réaliste comporte des faiblesses d'architecture, pas seulement des CVE. Vous êtes encouragés à en introduire **une ou deux**, à condition de les documenter :

- un conteneur attaché à deux réseaux « pour simplifier l'intégration » ;

- une règle de publication trop large sur l'OPNsense ;
- un compte de service réutilisé sur plusieurs conteneurs ;
- un accès prestataire permanent jamais révoqué ;
- une sauvegarde lisible depuis un réseau applicatif ;
- un outil d'administration qui répond aussi depuis le réseau utilisateurs.

Ces défauts font d'excellents chemins alternatifs — et d'excellents cas de détection.

4. Ce que la cellule défense doit pouvoir détecter

La sonde **Suricata** et l'interface **EveBox** sont fournies. Vos règles vont dans `/etc/suricata/rules/tp-local.rules`, avec des `sid` à partir de **9000001**.

Préparez-les **pendant** la phase de build : vous connaissez vos chemins d'attaque, donc vous connaissez les signaux. Couvrez au minimum ces quatre familles :

1. **Reconnaissance** — balayage de ports ou de répertoires sur les services publiés.
2. **Exploitation initiale** — motif d'exploitation caractéristique de vos N1 (requête d'injection, dépôt de webshell, en-tête forgé), réponse HTTP anormale.
3. **Mouvement latéral** — trafic entre réseaux Docker qui ne correspond à aucun flux applicatif légitime, authentification réussie depuis une source inhabituelle, usage d'un compte de service hors de son périmètre.
4. **Franchissement de la zone spécifique** — tout trafic entrant dans `net-spec` qui ne provient pas du conteneur passerelle légitime.

Squelette de règle locale :

```
alert http any any -> $HOME_NET any (msg:"KM - depot de webshell sur le portail"; \
  flow:established,to_server; http.method; content:"POST"; \
  http.uri; content:".php"; http.request_body; content:"eval("; \
  classtype:web-application-attack; sid:9000001; rev:1;)
```

Pour chaque règle, le rapport de détection doit contenir : l'auteur, l'objectif, le **prompt d'IA utilisé** le cas échéant, la règle produite, et une capture d'un déclenchement réel dans EveBox. Testez chaque règle en rejouant vous-mêmes l'attaque : une règle jamais déclenchée ne vaut rien.

Complétez les alertes réseau par les **journaux applicatifs des conteneurs** (`docker compose logs`) : c'est la seconde source de preuve du rapport.

5. Recette avant livraison

Voir l'*annexe 2 — Grille de recette*. Aucun service n'est considéré livré tant que :

1. il est joignable depuis le réseau prévu et **injoignable** depuis les autres ;
2. son exploit a été rejoué **depuis l'extérieur** par un membre du groupe qui ne l'a pas installé ;
3. son flag a été relu après placement, et **survit à un redémarrage complet de la pile** ;
4. son trafic est visible dans EveBox ;
5. il se relève seul après `docker compose down && docker compose up -d`.